

STATS 506 — Homework 2

Yuze Jin

2026-09-28

Repository: [DATASCI-506](#)

Problem 1 — Random Walk with Momentum

a. Simulation function and input checks

The four direction codes represent right, left, up, and down. The first direction is random. At each later step, a Bernoulli draw decides whether to keep the previous direction; otherwise a fresh random direction is used. The fresh choice can equal the previous direction.

```
#' Check inputs for a momentum walk
#' @param n A single positive integer number of steps.
#' @param p A single probability between 0 and 1.
#' @return Invisibly returns TRUE for valid inputs; otherwise gives an error.
check_walk_inputs <- function(n, p) {
  if (!is.numeric(n) || is.complex(n) || length(n) != 1 ||
      is.na(n) || !is.finite(n) || n < 1 || n %% 1 != 0 ||
      n > .Machine$integer.max) {
    stop("n must be a single positive integer.")
  }
  if (!is.numeric(p) || is.complex(p) || length(p) != 1 ||
      is.na(p) || !is.finite(p) || p < 0 || p > 1) {
    stop("p must be a single number between 0 and 1.")
  }
  invisible(TRUE)
}

#' Simulate a two-dimensional random walk with momentum
#' @param n A single positive integer number of steps.
```

```
#' @param p A single probability between 0 and 1.
#' @return A named vector with the final x and y coordinates.
momentum_walk <- function(n, p) {
  momentum_walk1(n, p)
}
```

Both implementations below call the input check, so the same restrictions apply to momentum_walk, momentum_walk1, and momentum_walk2.

b. Loop and apply implementations

In Version 1, a for loop records each chosen direction. In Version 2, lapply builds runs of consecutive steps in the same direction, then unlist joins the runs. In both versions, the final coordinates are the numbers of right minus left and up minus down steps.

```
#' Simulate a momentum walk with a for loop
#' @param n A single positive integer number of steps.
#' @param p A single probability between 0 and 1.
#' @return A named vector with the final x and y coordinates.
momentum_walk1 <- function(n, p) {
  check_walk_inputs(n, p)
  n <- as.integer(n)
  random_directions <- sample(1:4, n, replace = TRUE)
  keep_previous <- runif(n) < p

  directions <- integer(n)
  directions[1] <- random_directions[1]
  if (n > 1) {
    for (i in 2:n) {
      directions[i] <- if (keep_previous[i]) {
        directions[i - 1]
      } else {
        random_directions[i]
      }
    }
  }

  c(
    x = sum(directions == 1) - sum(directions == 2),
    y = sum(directions == 3) - sum(directions == 4)
  )
}
```

```

}

#' Simulate a momentum walk with lapply
#' @param n A single positive integer number of steps.
#' @param p A single probability between 0 and 1.
#' @return A named vector with the final x and y coordinates.
momentum_walk2 <- function(n, p) {
  check_walk_inputs(n, p)
  n <- as.integer(n)
  random_directions <- sample(1:4, n, replace = TRUE)
  keep_previous <- runif(n) < p

  new_direction_steps <- which(c(TRUE, !keep_previous[-1]))
  run_ends <- c(new_direction_steps[-1] - 1, n)
  runs <- lapply(seq_along(new_direction_steps), function(i) {
    run_length <- run_ends[i] - new_direction_steps[i] + 1
    rep(random_directions[new_direction_steps[i]], run_length)
  })
  directions <- unlist(runs, use.names = FALSE)

  c(
    x = sum(directions == 1) - sum(directions == 2),
    y = sum(directions == 3) - sum(directions == 4)
  )
}

set.seed(506)
momentum_walk1(10, 0.6)

```

```

x y
-4 4

```

```
momentum_walk2(10, 0.6)
```

```

x y
-1 5

```

```
momentum_walk1(1000, 0.6)
```

```

x y
34 -36

```

```
momentum_walk2(1000, 0.6)
```

```
      x    y  
-35 -63
```

The four positions vary because the calls above use successive random draws.

c. Reproducibility

`set.seed` sets R's random-number generator to the same starting state before each matched pair. Repeating a call with the same inputs and seed therefore produces the same sequence of draws and the same final position.

```
for (n in c(10, 1000)) {  
  set.seed(506)  
  first <- momentum_walk1(n, 0.6)  
  set.seed(506)  
  second <- momentum_walk1(n, 0.6)  
  cat("n =", n, "\n")  
  print(rbind(first = first, second = second))  
  stopifnot(identical(first, second))  
}
```

```
n = 10  
      x y  
first -4 4  
second -4 4  
n = 1000  
      x    y  
first 24 -38  
second 24 -38
```

For both 10 and 1,000 steps, the first and second rows match. Resetting the seed before each call makes R generate the same random choices in the same order, so the final positions are reproducible.

d. Does momentum matter in single walks?

```

momentum_values_single <- c(0, 0.5, 0.9)
set.seed(506)
single_endpoints <- t(vapply(
  momentum_values_single,
  function(probability) momentum_walk1(1000, probability),
  c(x = 0, y = 0)
))
single_walk_results <- data.frame(
  p = momentum_values_single,
  x = single_endpoints[, "x"],
  y = single_endpoints[, "y"],
  distance = sqrt(rowSums(single_endpoints^2))
)
single_walk_results

```

	p	x	y	distance
1	0.0	-33	-13	35.46830
2	0.5	-20	4	20.39608
3	0.9	43	55	69.81404

The final distances are 35.47 at $p = 0$, 20.4 at $p = 0.5$, and 69.81 at $p = 0.9$. They do not increase steadily in these three walks, although the $p = 0.9$ walk ends farthest away. One random walk per probability is not enough to establish the typical effect of momentum; part (e) uses repeated simulations.

e. Monte Carlo simulation

replicate runs 1,000 independent walks for each probability. Each call to `momentum_walk1` uses fresh random draws. The columns of endpoints contain the x and y coordinates from one walk.

```

momentum_values <- c(0, 0.25, 0.5, 0.75, 0.9)
set.seed(506)
mean_distances <- vapply(momentum_values, function(probability) {
  endpoints <- replicate(
    1000,
    momentum_walk1(1000, probability)
  )
  mean(sqrt(colSums(endpoints^2)))
}, numeric(1))

```

```
monte_carlo_results <- data.frame(
  p = momentum_values,
  mean_distance = round(mean_distances, 2)
)
monte_carlo_results
```

	p	mean_distance
1	0.00	27.50
2	0.25	35.94
3	0.50	47.82
4	0.75	76.27
5	0.90	121.97

The mean final distance rises from 27.5 at $p = 0$ to 121.97 at $p = 0.9$, increasing across all five probabilities in the table. Momentum makes consecutive steps more likely to point in the same direction, so a walk tends to end farther from the origin.

f. Computational performance

The two functions use the same random draws and direction rules. `microbenchmark` times each call repeatedly; the table compares median elapsed times in milliseconds.

```
library(microbenchmark)
set.seed(506)
benchmark_1000 <- microbenchmark(
  loop = momentum_walk1(1000, 0.6),
  apply = momentum_walk2(1000, 0.6),
  times = 20L
)
benchmark_100000 <- microbenchmark(
  loop = momentum_walk1(100000, 0.6),
  apply = momentum_walk2(100000, 0.6),
  times = 10L
)

benchmark_results <- rbind(
  data.frame(n = 1000, summary(benchmark_1000, unit = "ms")[
    , c("expr", "median")
  ]),
  data.frame(n = 100000, summary(benchmark_100000, unit = "ms")[
```

```

    , c("expr", "median")
  ])
)
names(benchmark_results) <- c("n", "implementation", "median_ms")
benchmark_results$n <- format(
  benchmark_results$n, scientific = FALSE, trim = TRUE
)
benchmark_results$median_ms <- round(benchmark_results$median_ms, 3)
benchmark_results

```

	n	implementation	median_ms
1	1000	loop	0.051
2	1000	apply	0.197
3	100000	loop	4.695
4	100000	apply	20.803

On this machine, the loop is faster for both step counts. The difference in median time is much larger at 100,000 steps in absolute milliseconds. The apply version creates many separate vectors before joining them, whereas the loop directly assigns into a preallocated vector. Exact timing depends on the machine.

Problem 2 — Monte Carlo Simulation of Campus Bus Arrivals

Each row of a matrix represents one day, and each column represents one hour in that time period. `rowSums` adds the hourly Poisson draws for each day. For the two peak hours, the standard deviations are the square roots of the stated variances, 4 and 5. I use the Normal draws as specified, without rounding them to integers.

```

n_days <- 10000
set.seed(506)
morning <- matrix(rpois(n_days * 3, lambda = 4), nrow = n_days)
midday <- matrix(rpois(n_days * 6, lambda = 15), nrow = n_days)
evening <- matrix(rpois(n_days * 6, lambda = 9), nrow = n_days)
daily_passengers <- (
  rowSums(morning) +
  rnorm(n_days, mean = 45, sd = sqrt(16)) +
  rowSums(midday) +
  rnorm(n_days, mean = 55, sd = sqrt(25)) +
  rowSums(evening)
)

```

```
estimated_daily_mean <- mean(daily_passengers)
round(estimated_daily_mean, 2)
```

```
[1] 256.2
```

The Monte Carlo estimate is 256.2 passengers per day. This is close to the theoretical mean of $3(4) + 45 + 6(15) + 55 + 6(9) = 256$. The simulation generates all 10,000 days without a loop.

Problem 3 — Linear Regression

The data and variable definitions come from the [TidyTuesday YouTube dataset](#).

a. Data preparation and de-identification

I retain only year, the seven binary commercial characteristics, and the five engagement counts. This removes brand, URLs, title, description, channel, publication data, video identifiers, and other unused metadata.

```
youtube <- read.csv(
  "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2021/2021-03-02",
)

characteristics <- c(
  "funny", "show_product_quickly", "patriotic", "celebrity",
  "danger", "animals", "use_sex"
)

engagement_counts <- c(
  "view_count", "like_count", "dislike_count",
  "favorite_count", "comment_count"
)

youtube_clean <- youtube[c("year", characteristics, engagement_counts)]
dim(youtube_clean)
```

```
[1] 247 13
```

After removing the identifying fields, the dataset has 247 rows and 13 columns. The retained columns are the variables needed for the following analyses.

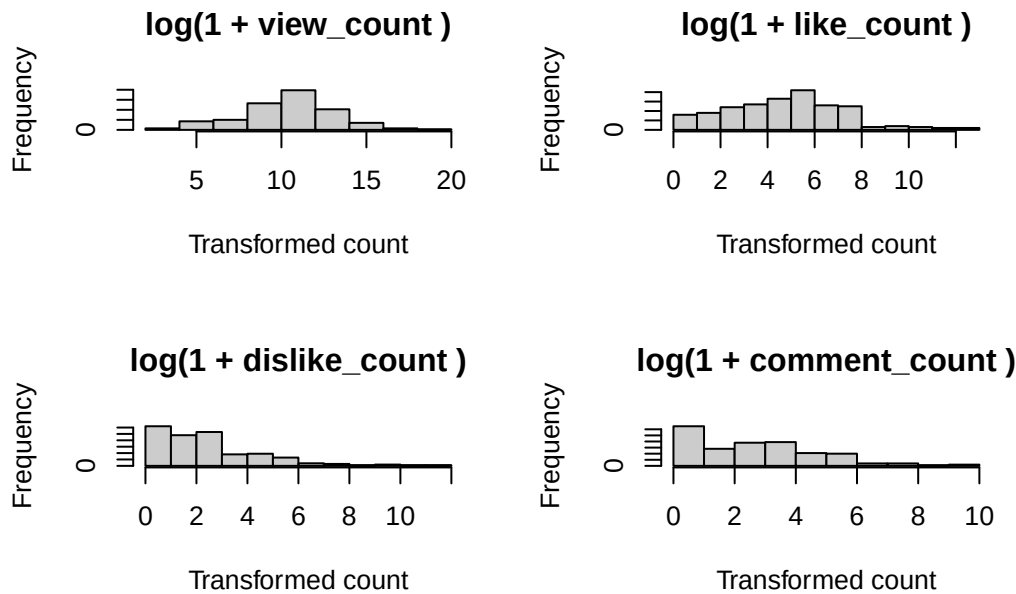
b. Exploring potential outcomes

The table reports missing values, zeros, and key quantiles for each count. A maximum far above the median indicates a long right tail.

```
distribution_table <- do.call(rbind, lapply(engagement_counts, function(name) {
  values <- youtube_clean[[name]]
  observed <- values[!is.na(values)]
  data.frame(
    outcome = name,
    missing = sum(is.na(values)),
    zero = sum(observed == 0),
    median = median(observed),
    q75 = unname(quantile(observed, 0.75)),
    maximum = max(observed)
  )
}))
distribution_table
```

	outcome	missing	zero	median	q75	maximum
1	view_count	16	0	41379	170015.50	176373378
2	like_count	22	9	130	527.00	275362
3	dislike_count	22	47	7	24.00	92990
4	favorite_count	16	231	0	0.00	0
5	comment_count	25	41	10	50.75	9190

```
par(mfrow = c(2, 2))
for (name in engagement_counts[engagement_counts != "favorite_count"]) {
  hist(
    log(1 + youtube_clean[[name]]),
    main = paste("log(1 +", name, ")"),
    xlab = "Transformed count",
    col = "gray80"
  )
}
```



```
par(mfrow = c(1, 1))
```

Each engagement variable falls into one of the assignment's three categories:

Variable	Category	Reason
View count	2 — transform	The maximum is far above the median, showing a long right tail.
Like count	2 — transform	The counts are right-skewed and include zeros.
Dislike count	2 — transform	The counts have a long right tail and many zeros.
Favorite count	3 — do not use	Every observed value is zero, so there is no outcome variation to model.
Comment count	2 — transform	The counts have a long right tail and many zeros.

No variable is suitable for category 1 (use unchanged). I use $\log(1 + \text{count})$ for the four variables in category 2; adding 1 keeps zero counts defined. The transformed dislike and comment counts still have a large concentration near zero and a right tail; the transformation does not make

them approximately normal. A linear model does not require the outcome's overall distribution to be normal, but inference from its p-values depends on the model errors. I keep these two as exploratory transformed outcomes and would check residuals before relying on their p-values. Missing values remain missing and are omitted separately by each model.

```
regression_outcomes <- c(
  "log_view_count", "log_like_count",
  "log_dislike_count", "log_comment_count"
)
for (name in engagement_counts[engagement_counts != "favorite_count"]) {
  youtube_clean[[paste0("log_", name)]] <- log(1 + youtube_clean[[name]])
}
```

c. Separate linear regression models

Each model includes one binary characteristic and year as a continuous covariate. The coefficient compares commercials with and without that characteristic, conditional on year, on the $\log(1 + \text{count})$ scale.

```
model_results <- do.call(rbind, lapply(regression_outcomes, function(outcome) {
  do.call(rbind, lapply(characteristics, function(characteristic) {
    model_data <- data.frame(
      y = youtube_clean[[outcome]],
      characteristic = youtube_clean[[characteristic]],
      year = youtube_clean$year
    )
    model <- lm(y ~ characteristic + year, data = model_data)
    coefficient <- summary(model)$coefficients[
      "characteristicTRUE",
    ]
    data.frame(
      outcome = outcome,
      characteristic = characteristic,
      n = nobs(model),
      estimate = unname(coefficient["Estimate"]),
      p_value = unname(coefficient["Pr(>|t|)"])
    )
  })))
rownames(model_results) <- NULL
data.frame(
  outcome = model_results$outcome,
```

```

characteristic = model_results$characteristic,
n = model_results$n,
estimate = round(model_results$estimate, 3),
p_value = signif(model_results$p_value, 3)
)

```

	outcome	characteristic	n	estimate	p_value
1	log_view_count	funny	231	0.530	0.2050
2	log_view_count	show_product_quickly	231	0.209	0.6010
3	log_view_count	patriotic	231	0.137	0.7840
4	log_view_count	celebrity	231	-0.019	0.9630
5	log_view_count	danger	231	0.667	0.0941
6	log_view_count	animals	231	-0.173	0.6470
7	log_view_count	use_sex	231	-0.296	0.4910
8	log_like_count	funny	225	0.360	0.3400
9	log_like_count	show_product_quickly	225	0.168	0.6400
10	log_like_count	patriotic	225	0.493	0.2900
11	log_like_count	celebrity	225	0.263	0.4780
12	log_like_count	danger	225	0.625	0.0803
13	log_like_count	animals	225	0.025	0.9430
14	log_like_count	use_sex	225	-0.325	0.4000
15	log_dislike_count	funny	225	0.041	0.8960
16	log_dislike_count	show_product_quickly	225	0.243	0.4210
17	log_dislike_count	patriotic	225	0.670	0.0856
18	log_dislike_count	celebrity	225	-0.283	0.3620
19	log_dislike_count	danger	225	0.185	0.5380
20	log_dislike_count	animals	225	-0.074	0.7970
21	log_dislike_count	use_sex	225	-0.389	0.2290
22	log_comment_count	funny	222	0.047	0.8790
23	log_comment_count	show_product_quickly	222	0.346	0.2470
24	log_comment_count	patriotic	222	0.515	0.1690
25	log_comment_count	celebrity	222	0.228	0.4560
26	log_comment_count	danger	222	0.106	0.7210
27	log_comment_count	animals	222	-0.192	0.4990
28	log_comment_count	use_sex	222	-0.349	0.2740

```

significant_results <- model_results[model_results$p_value < 0.05, ]
significant_results

```

```

[1] outcome      characteristic n      estimate      p_value
<0 rows> (or 0-length row.names)

```

```

view_results <- model_results[
  model_results$outcome == "log_view_count",
]
largest_positive_view <- view_results[
  which.max(view_results$estimate),
]
largest_positive_view

```

	outcome	characteristic	n	estimate	p_value
5	log_view_count	danger	231	0.6667808	0.09406479

At the $\alpha = 0.05$ level, **none** of the seven characteristics is significantly associated with any of the four outcomes: transformed views, likes, dislikes, or comments. Therefore, there are no significant positive or negative associations to list. danger has the largest positive estimated association with transformed view count (coefficient 0.667), but its p-value of 0.094 is above 0.05. These separate observational models do not establish causation.

d. Regression coefficients by matrix algebra

Danger also has the smallest view-count p-value in part (c). I use the same complete cases and the same design matrix columns for the matrix calculation and lm comparison.

```

selected_characteristic <- view_results$characteristic[
  which.min(view_results$p_value)
]
regression_data <- data.frame(
  y = youtube_clean$log_view_count,
  characteristic = youtube_clean[[selected_characteristic]],
  year = youtube_clean$year
)
regression_data <- regression_data[complete.cases(regression_data), ]

X <- cbind(
  "(Intercept)" = 1,
  as.integer(regression_data$characteristic),
  year = regression_data$year
)
colnames(X)[2] <- selected_characteristic
y <- regression_data$y

```

```

beta_matrix <- solve(t(X) %*% X, t(X) %*% y)
comparison_model <- lm(y ~ characteristic + year, data = regression_data)
beta_lm <- unname(coef(comparison_model))
coefficient_comparison <- data.frame(
  term = colnames(X),
  matrix_estimate = as.vector(beta_matrix),
  lm_estimate = beta_lm,
  absolute_difference = abs(as.vector(beta_matrix) - beta_lm)
)
selected_characteristic

```

```
[1] "danger"
```

```
coefficient_comparison
```

	term	matrix_estimate	lm_estimate	absolute_difference
1	(Intercept)	-31.39149243	-31.39149243	4.247163e-10
2	danger	0.66678076	0.66678076	2.325917e-13
3	year	0.02065641	0.02065641	2.112442e-13

The selected characteristic is danger. The design matrix has columns for the intercept, that characteristic, and year. The largest absolute difference between the matrix estimates and `lm()` estimates is 4.25×10^{-10} , so they agree up to numerical precision. The small differences arise because the normal-equation calculation and `lm()` use different numerical methods.

Sources and assistance

OpenAI Codex assisted with drafting and checking the R code and explanations. The YouTube dataset and its variable descriptions are from [TidyTuesday](#).